InstallGen AI – Prompt Engineering Document

Prompt 1: Understanding the Problem and Designing the Solution

I want to build an AI-powered Service Desk application that can automate software installation requests.

The main goal is to reduce repetitive tickets where users ask support teams how to install software such as Docker, Python, Node.js, Git, Java, MySQL, and VS Code.

The application should understand the user's request, collect environment details such as operating system and version, generate installation scripts, validate compatibility, provide troubleshooting guidance, and generate reports.

Suggest a complete system architecture, workflow, database design, API structure, and technology stack for this project.

Prompt 2: Designing the Multi-Agent Workflow

Help me design a multi-agent architecture for an installation automation platform.

The system should have specialized agents responsible for:

- Understanding installation requests
- Collecting environment information
- Retrieving installation procedures
- Generating scripts
- Validating compatibility
- Troubleshooting errors
- Generating reports

Explain how each agent works, what input it receives, what output it produces, and how agents communicate with one another.

Prompt 3: Backend Development

Generate a FastAPI backend for the InstallGen AI platform.

The backend should:

- Use FastAPI and SQLite
- Follow a modular folder structure
- Use SQLAlchemy for database operations
- Provide REST APIs for installation requests, script generation, reporting, troubleshooting, and AI assistance
- Include proper validation and error handling

Create production-style code with clear separation of concerns.

Prompt 4: Creating the Installation Knowledge Base

Create a knowledge base containing installation procedures for common software such as Docker, Python, Node.js, Git, Java, MySQL, and VS Code.

The instructions should support:

- Windows
- Ubuntu Linux
- CentOS
- macOS

Organize the data so that AI agents can easily retrieve and use it when generating installation scripts.

Prompt 5: Building the Script Generation Agent

Create an intelligent script generation agent.

The agent should accept:

- Software name
- Software version
- Operating system
- Operating system version
- Package manager

Based on the provided environment, generate installation commands, verification steps, and rollback procedures.

Support Windows PowerShell, Linux Shell, and macOS Terminal commands.

Prompt 6: Compatibility Validation Agent

Create a validation agent that checks whether a requested software version is compatible with the selected operating system and environment.

The agent should:

- Identify unsupported combinations
- Display warnings
- Suggest alternatives when needed
- Return a validation result before script generation

Prompt 7: Troubleshooting Agent

Create an AI troubleshooting agent that helps users solve installation issues.

The agent should analyze installation error messages and provide:

- Possible root causes
- Recommended solutions
- Step-by-step troubleshooting guidance
- Confidence score for each recommendation

Use Gemini AI to improve reasoning and explanation quality.

Prompt 8: Report Generation Agent
Create a documentation and reporting agent.

After script generation, the agent should prepare a report containing:

- Installation request details
- Environment information
- Generated installation script
- Validation results
- Troubleshooting guidance
- Timestamp

The report should be exportable as a professional PDF document.

Prompt 9: Frontend Development
Build a modern frontend for InstallGen AI using React and Tailwind CSS.

The application should include:

- Dashboard
- Installation Request Form
- Generated Script Viewer
- Troubleshooting Page
- Reports Page
- AI Assistant Page

The interface should be responsive, professional, easy to use, and suitable for an enterprise environment.

Prompt 10: AI Assistant

Create an AI-powered service desk assistant using Gemini AI.

The assistant should answer questions related to:

- Software installation
- Command explanations
- Compatibility issues
- Troubleshooting support

Provide a chat-based interface that integrates with the backend APIs.

Prompt 11: Final Integration

Integrate the frontend, backend, database, AI agents, and Gemini API into a single application.

Ensure that:

- All agents work together through a clear orchestration pipeline
- Installation requests flow correctly through each stage
- Scripts, reports, and troubleshooting recommendations are generated successfully
- Frontend and backend communication is fully functional

Prepare the application for demonstration and testing.

Development Approach

The project was developed using a prompt-driven engineering approach.

The implementation was completed in phases:

1. Requirement Analysis
2. System Architecture Design
3. Multi-Agent Workflow Design
4. Backend Development
5. Knowledge Base Development
6. Script Generation Logic
7. Troubleshooting Engine
8. Report Generation
9. Frontend Development
10. AI Integration
11. End-to-End Testing

Generative AI was used as a development accelerator, while the overall architecture, workflow design, feature selection, integration, testing, and project customization were performed throughout the development process.